

Embedded Artificial Intelligence

Cours et méthodes : cours 01 et 02, TP 1 et 2

BMEVIMIMA22, Béla Pataki, automne 2026. Rédigé le 2026-09-28.

À retenir

Midterm	50 points, le 2026-10-29 de 18h à 20h
Rattrapage	le 2026-11-09 de 18h à 20h : ton unique essai (Wuhan)
Signature	40 % au midterm, soit 20 points sur 50 ; sans elle, pas d'examen
Format	hybride, QCM et questions ouvertes (enseignant, 2026-09-24)
Quiz en cours	1 à 4 par cours, 1/3 de point par bonne réponse ; plafond À VÉRIFIER
Examen final	écrit, 50 points, slides et lectures complémentaires
Note	examen + $0,2 \times$ midterm (max 10) + quiz
Réussite	40 % de (50 + 10), soit 24 points

Source : fiche CLAUDE.md de la matière ; 01_Intro, slide 3

Exemple. Midterm à 30/50 : $0,2 \times 30 = 6$ points ; sans quiz, il reste $24 - 6 = 18$ points sur 50 à obtenir à l'examen final.

Piège

Le rattrapage du 9 novembre est ton seul essai pour la signature. Qu'il compte aussi comme midterm dans la note finale : confirmation écrite **À VÉRIFIER**.

Périmètre. Supports disponibles : 01_Intro (36 slides), 02_Challenges (34 pages), notebooks des TP 1 et 2 avec solutions. La suite du programme (concept 20) n'est pas développée, faute de support. Limite du midterm : **À VÉRIFIER**.

Sources. Slide (01_Intro), page du PDF (02_Challenges), cellule (notebooks). [rappel général] marque une explication générale absente des supports. Calculs vérifiés par `calc_embai.py`.

Table des matières

1	Systèmes embarqués et informatique généraliste	4
1.1	Trois exemples, trois profils de contraintes	4
2	Qu'est-ce que l'IA ?	5
2.1	Quatre approches (slide 14)	5
2.2	Repères historiques (slides 15 et 16)	6
2.3	Domaines d'application (slide 17)	6
3	IA, ML, DL, IA générative ; modèle ou données	7
3.1	Méthode fondée sur un modèle ou sur les données (slides 20 et 21)	7
3.2	La qualité des données (slides 22 et 23)	7
4	Ordres de grandeur : le coût des grands modèles	8
5	IA embarquée : AIoT, TinyML, EdgeAI	9
6	Complexité algorithmique et notation O	10
6.1	Le problème des pièces (slides 32 et 33)	11
6.2	Algorithmes de tri (slide 34)	12
7	Apprentissage automatique : les principes	12
8	Défis 1 à 5 : entraînement, taille, énergie, vitesse, encombrement	14
8.1	Défi 1, temps d'entraînement (p. 10 et 11)	14
8.2	Défi 2, taille du modèle (p. 12 à 14)	15
8.3	Défi 3, efficacité énergétique (p. 15)	15
8.4	Défi 4, vitesse d'inférence (p. 16)	16
8.5	Défi 5, taille et poids physiques (p. 17)	16

9 Défis 6 à 8 : sûreté, confiance, gouvernance	16
9.1 Défi 6, sûreté (p. 18)	16
9.2 Défi 7, confiance (p. 19 à 22)	16
9.3 Défi 8, gouvernance (p. 23)	17
10 Où calculer : cloud ou local	18
11 Où part l'énergie	19
12 Matériel, premières applications, capteurs intelligents	20
13 Représentation des nombres et quantification	21
13.1 Taille mémoire d'un modèle	22
14 TP 1 : Python	22
14.1 Exercices du TP et solutions	23
15 TP 1 : NumPy et Matplotlib	24
15.1 Matplotlib	25
16 TP 2 : régression linéaire, solution analytique	26
17 TP 2 : régression polynomiale et régularisation	28
18 TP 2 : jeu de données réel et RMSE	29
19 TP 2 : régression logistique	30
20 Ce qui reste au programme	32
21 Méthode d'examen	33
22 Formulaire	35

1. Systèmes embarqués et informatique généraliste

Source : 01_Intro, slides 6 à 9

Critère	Ordinateur généraliste	Système embarqué
But	large éventail de tâches (PC, serveurs)	optimisé pour une tâche précise, donc plus efficace pour elle
Contraintes	peu contraint	mémoire, puissance de calcul, temps de réponse, consommation limités : conception matérielle et logicielle spécialisée
Interface	riche, interaction directe	souvent aucune, ou minimale
Durée de vie	remplacé en quelques années	souvent dix ans ou plus (aéronautique, automobile)
Logiciel	Windows, macOS, Linux	OS temps réel, ou aucun OS

Trois exemples, trois profils de contraintes

Exemple	Important	Non critique
Maison connectée (slide 7)	communication, fusion d'informations, consommation (d'une partie au moins) des capteurs	mémoire, puissance de calcul, temps de réponse
Voiture autonome (slide 8)	puissance de calcul, temps de réponse, communication, fusion d'informations	mémoire, consommation des capteurs, poids
Surveillance environnementale (slide 9)	communication, fusion, consommation des appareils (batterie, « !!! »), petit, léger et robuste	mémoire, puissance de calcul, temps de réponse

Capteurs cités. Maison : portes et fenêtres, température, consommation électrique, mouvement, fumée, fuite d'eau ; commande à distance (mobile, voix) et minuterie pour économiser l'énergie. Voiture : caméras, LIDAR, ultrasons, radar, centrale inertielle, GPS ; commande automatique ou humaine, vocale

ou physique. Forêt : de simples capteurs audio, un système de détection et une alarme contre les motards qui traversent une forêt protégée (sécurité, bruit, dégâts).

Piège

Aucun des trois n’a « tout » critique. La voiture ne se soucie pas de la consommation des capteurs; le capteur en forêt ne se soucie pas du temps de réponse. Une question type : « quelles contraintes sont critiques pour telle application ? ». Raisonner depuis l’alimentation (secteur, batterie de voiture, pile) et depuis la gravité d’une réponse tardive.

2. Qu’est-ce que l’IA ?

Source : 01_Intro, slides 10 à 17

L’intelligence est une notion floue; les chercheurs en IA utilisent surtout la **rationalité** : choisir la meilleure action pour atteindre un but, selon des critères à optimiser et avec les ressources disponibles (slide 11). Définition du Parlement européen (slide 13) : capacité d’une machine à montrer des aptitudes humaines comme le raisonnement, l’apprentissage, la planification et la créativité.

Test de Turing (1950, slide 13) : un juge humain dialogue avec une personne et une machine; s’il ne sait pas dire laquelle est la machine, celle-ci réussit le test.

Quatre approches (slide 14)

	Comme un humain	Rationnellement
Penser	penser comme un humain	penser rationnellement
Agir	agir comme un humain	agir rationnellement

La slide entoure la colonne « rationnellement » : « humaine manner ≠ rational ». Approche d’ingénieur : un but fixé d’avance, atteint avec le moins d’effort, de temps ou d’énergie. Les buts s’expriment en utilité des résultats; le monde est incertain, donc on utilise l’utilité espérée; être rationnel, c’est **maximiser l’utilité espérée**. La voie humaine n’est pas forcément la

meilleure : les avions à ailes battantes ont échoué, la roue et la route vont plus vite que les jambes.

Repères historiques (slides 15 et 16)

1950	Alan Turing, test de Turing
1956	atelier de Dartmouth : naissance de l'IA
1966	premier chatbot, ELIZA
1970 à 1980	hiver de l'IA après le premier boom, recherche en pause
1997	Deep Blue (IBM) bat Garry Kasparov aux échecs
2011	Watson (IBM) gagne à Jeopardy ; Siri sur l'iPhone 4S
2012	AlexNet, CNN de reconnaissance d'images
2014	Amazon Alexa ; le chatbot Eugene Goostman passe le test de Turing
2016	Tay suspendu après 16 heures pour ses messages racistes et sexistes
2017	AlphaGo bat Ke Jie au go
2018	premier GPT ; lignes directrices éthiques de l'UE pour une IA digne de confiance
2020	GPT-3

Domaines d'application (slide 17)

La slide encadre trois applications comme **typiquement embarquées** : conduite autonome (Tesla), prédiction du trafic, surveillance environnementale. Trois autres sont **généralistes ou embarquées** : maintenance prédictive, reconnaissance de formes et d'images, reconnaissance vocale. Les autres (recommandation de produits, trading, assistant personnel, filtrage de spam et de malware, découverte de médicaments et diagnostic, programmation) ne sont pas encadrées. Slide 19 : parmi les métiers qu'une IA pourrait faire mieux que l'humain (étude BBC), la slide marque comme embarqués « plier le linge », « chauffeur routier » et « chirurgien ».

3. IA, ML, DL, IA générative ; modèle ou données

Source : 01_Intro, slides 18 à 23

À retenir

- **IA** : techniques qui permettent à un ordinateur de résoudre des problèmes « difficiles », de s'adapter, d'apprendre, d'explorer.
- **Apprentissage automatique (ML)** : sous-ensemble de l'IA ; des algorithmes prédisent à partir de données. L'information est donnée sous forme d'*exemples*, pas de lois, de règles ou de formules.
- **Apprentissage profond (DL)** : sous-ensemble du ML ; réseaux de neurones à nombreuses couches qui extraient eux-mêmes les caractéristiques des données brutes.
- **IA générative** : sous-ensemble du DL qui produit du contenu (texte, images, code), entraîné sur de très grands jeux de données.

Schéma de la slide 18. Quatre ensembles emboîtés : $IA \supset ML \supset DL \supset IA \text{ générative}$. Une ellipse « Embedded AI » traverse les cercles IA, ML et DL : l'IA embarquée n'est pas une couche de plus, elle recoupe plusieurs niveaux.

Méthode fondée sur un modèle ou sur les données (slides 20 et 21)

Avec modèle. On connaît la structure : le circuit est linéaire, sa fonction de transfert a une forme connue, par exemple un facteur $1/(1 + j\omega R_3 C)$. On mesure V_1 et V_2 à plusieurs fréquences et on calcule R_1 à R_4 , L et C . Questions : le modèle est-il juste ? comment obtenir les valeurs ? précision et bruit de mesure.

Sans modèle (ML). La boîte est noire : $V_2 = f(V_1(\omega))$. On n'a que des données (un tableau de V_1 , ω , V_2 mesurés). Questions : combien de données faut-il ? comment passer des données à la prédiction ? précision et bruit de mesure. Même but dans les deux cas : prédire V_2 pour tout V_1 et tout ω .

La qualité des données (slides 22 et 23)

Le ML apprend sur des exemples « sans être explicitement programmé » : l'algorithme d'entraînement est général, pas propre au problème. Il faut donc des données pertinentes, non biaisées et peu bruitées ; nettoyer les données

(réduction du bruit, détection et traitement des valeurs aberrantes) est capital, même si le cours ne le traite pas faute de temps. Chiffre cité (Gartner) : d’ici 2027, les organisations qui soignent la sémantique de leurs données pourraient gagner jusqu’à 80 % de précision et réduire leurs coûts de 60 %.

Piège

« Garbage in, garbage out » (slide 23) : le meilleur algorithme du monde, avec des données non pertinentes, bruitées ou biaisées, produit un résultat sans valeur.

4. Ordres de grandeur : le coût des grands modèles

Source : 01_Intro, slides 24 à 26

Modèle	Année	Paramètres (slide 24)
GPT-1	2018	117 millions
GPT-2	2019	1,5 milliard
GPT-3	2020	175 milliards
GPT-3.5	2022	175 milliards
GPT-4.5	2023	1,7 billion ($1,7 \cdot 10^{12}$)

Entraînement (slide 25)	GPT-3	GPT-4
Durée	34 jours	100 jours
Énergie	1 287 000 kWh	62 318 800 kWh
Coût	168 597 \$	8 163 763 \$

Calculs de la slide 25

GPT-4 : 62 318 800 kWh = $6,23 \cdot 10^{10}$ Wh, à comparer aux $\approx 6,1 \cdot 10^{10}$ Wh que consomme la Hongrie en une demi-journée : ratio 1,02. GPT-4 contre GPT-3 : $62\,318\,800 / 1\,287\,000 \approx 48$ fois plus d’énergie. Prix implicite : $168\,597 / 1\,287\,000 = 0,131$ \$ par kWh, le tarif indiqué en bas de la slide.

Calcul de la slide 26 : l'inférence

$2 \cdot 10^9$ requêtes par jour à 0,34 Wh chacune : $2 \cdot 10^9 \times 0,34 = 0,68 \cdot 10^9$ Wh par jour. La Hongrie consomme $0,85 \cdot 10^9$ Wh en 10 minutes : ChatGPT consomme chaque jour l'équivalent de $10 \times 0,68/0,85 = 8$ minutes de consommation du pays.

Répartition de l'énergie d'une requête (camembert de la slide 26) : 75 % calcul, 10 % refroidissement, 10 % gestion des données, 5 % infrastructure.

À retenir

Message du cours : l'IA « sur ordinateur » coûte énormément d'énergie, à l'entraînement comme à l'inférence. En embarqué, l'énergie est limitée (batterie) : il faut d'autres modèles et d'autres techniques.

5. IA embarquée : AIoT, TinyML, EdgeAI

Source : 01_Intro, slides 27 à 30

Mots-clés du domaine (slide 27) : AIoT, TinyML, EdgeAI, MLsensors. Algorithmes très optimisés et à faible empreinte, y compris pour l'audio et la vidéo. Applications « toujours actives » : il faut minimiser la consommation, et elles posent un problème de vie privée. Matériel dédié basse consommation : microcontrôleurs et DSP ultra basse consommation, couplés étroitement aux capteurs, sous-systèmes d'appareils plus complexes.

Techniques TinyML annoncées pour plus tard (slide 28) : **quantification** (réduire la précision numérique des poids), **élagage** (*pruning*, supprimer des poids ou neurones inutiles), algorithmes spécialisés. Chaîne de la figure : entraînement, distillation, quantification, encodage, compilation ; le modèle TensorFlow est converti au format `.tflite`, puis en tableau C chargé sur le microcontrôleur.

Avantages d'EdgeAI (slide 29) : latence réduite, confidentialité (pas d'envoi vers un serveur), économie de bande passante, résilience (fonctionne sans connexion). Exemple de matériel : kit NVIDIA Jetson Nano (CPU ARM Cortex-A57, GPU Maxwell, 4 Go de RAM).

	TinyML (slide 28)	EdgeAI (slide 29)
Idée	ML sur matériel très contraint : microcontrôleurs	exécuter l'IA sur un appareil local, près de la source des données
Cibles	IoT, objets portés, edge computing	IoT, smartphones, serveurs de périphérie, puces IA
Applications	objets connectés, wearables	véhicules autonomes, caméras, drones, capteurs industriels
Outils	TensorFlow Lite for Microcontrollers	TensorFlow Lite, ONNX Runtime, NVIDIA Jetson
Défis	mémoire, calcul, énergie	ressources limitées face au cloud ; compression et optimisation du modèle

Ce que le cours va traiter (slide 30) : les ressources limitent la complexité de l'algorithme (consommation, temps de réponse, mémoire) ; on verra des méthodes d'IA importantes (pas toutes), les options matérielles et la communication. Les méthodes et le matériel changent vite, mais les algorithmes de base et les grandes tendances restent.

6. Complexité algorithmique et notation O

Source : 01_Intro, slides 31 à 35

Complexité en temps : nombre d'exécutions des instructions, pas la durée mesurée (qui dépend du compilateur et du processeur). **Complexité en espace** : mémoire totale nécessaire. Les deux s'expriment en fonction de la taille N de l'entrée ; on distingue cas moyen et pire cas (slides 31 et 34).

À retenir

Définition (slide 31). $T(N)$ est $O(F(N))$ s'il existe des constantes positives c et N_0 telles que $T(N) \leq c F(N)$ pour tout $N > N_0$: une borne supérieure pour les grands N .

O : borne supérieure ; Ω : borne inférieure ; Θ : ordre exact (slide 35).

Prouver que $3N^2 + 5N + 2$ est $O(N^2)$ (méthode générale)

On choisit $c = 4$. Il faut $3N^2 + 5N + 2 \leq 4N^2$, soit $N^2 - 5N - 2 \geq 0$. La racine positive vaut $(5 + \sqrt{33})/2 \approx 5,37$: l'inégalité tient pour tout entier $N \geq 6$. Donc $c = 4$ et $N_0 = 5$ conviennent (condition $N > N_0$). Tout couple valable suffit ; seule l'existence compte.

Les constantes (slide 35) : la notation O les ignore, car elle ne décrit que la croissance à long terme ; mais en pratique elles comptent. Distinguer « sont-elles pertinentes ? » (oui) et « O s'en soucie-t-elle ? » (non).

Le problème des pièces (slides 32 et 33)

N pièces, une fausse plus légère, une balance à deux plateaux. Force brute : $N - 1$ pesées, $O(N)$. La slide donne comme solution $O(\log_2 N)$, et mieux $O(\log_3 N)$.

Méthode par tiers (méthode générale)

Répartir les pièces en trois groupes aussi égaux que possible et peser deux groupes de même taille. Le plateau le plus léger contient la fausse pièce ; en cas d'équilibre, elle est dans le troisième groupe. Chaque pesée divise les candidats par 3 : il faut $\lceil \log_3 N \rceil$ pesées. Avec 3 pièces (indice de la slide), une seule pesée suffit : pièce 1 contre pièce 2.

N	force brute $N - 1$	moitiés $\lceil \log_2 N \rceil$	tiers $\lceil \log_3 N \rceil$
9	8	4	2
27	26	5	3
100	99	7	5
1000	999	10	7

Piège

$O(\log_2 N)$ et $O(\log_3 N)$ sont la même classe : $\log_2 N = \log_2 3 \times \log_3 N$ avec $\log_2 3 \approx 1,585$, un facteur constant. Le nombre réel de pesées diffère, la classe O non. C'est exactement la remarque de la slide 35 sur les constantes.

Algorithmes de tri (slide 34)

	meilleur	moyen	pire	mémoire (pire)
Quicksort	$O(N \log N)$	$O(N \log N)$	$O(N^2)$	$O(N \log N)$
Mergesort	$O(N \log N)$	$O(N \log N)$	$O(N \log N)$	$O(N)$
Tri à bulles	$O(N)$	$O(N^2)$	$O(N^2)$	$O(1)$

$O(1)$ en mémoire : tri en place, aucune mémoire supplémentaire proportionnelle à N .

Écart concret pour $N = 1000$

$N^2 = 10^6$ contre $N \log_2 N \approx 9\,966$: environ 100 fois plus d’opérations. Pour $N = 10^6$, le rapport $N/\log_2 N$ dépasse 50 000. Sur la courbe de la slide 33, $\log_2 N$ reste sous 7 pour $N \leq 100$ alors que N monte à 100.

Une passe de tri à bulles sur l’exemple de la slide 34

Liste [3,9; -2; 1; 7; 12; 5; -2,37]. On compare les voisins de gauche à droite et on échange s’ils sont mal rangés. Passe 1 : quatre échanges, résultat [-2; 1; 3,9; 7; 5; -2,37; 12] : le maximum 12 est en place. Le -2,37 ne recule que d’une case par passe : il faut 6 passes avec échanges, puis une 7^e sans échange pour conclure.

Piège

La mémoire d’un quicksort : la slide indique $O(N \log N)$. Les références usuelles donnent $O(\log N)$ en moyenne et $O(N)$ au pire selon l’implémentation. **À VÉRIFIER** ; à l’examen, répondre avec la valeur de la slide. Pour l’embarqué, retenir le compromis : le tri à bulles est lent mais ne demande aucune mémoire en plus.

7. Apprentissage automatique : les principes

Source : 02_Challenges, p. 4 à 8 du PDF

LIA aujourd'hui (cloud) est	Un système embarqué a
gourmande en calcul	un processeur moins performant
gourmande en mémoire	une mémoire limitée
gourmande en données (entraînement)	une énergie limitée (batterie)
gourmande en énergie	des contraintes temps réel, un temps de réaction parfois critique
lente pour beaucoup de problèmes temps réel	une bande passante limitée et une communication peu fiable

À retenir

- **Supervisé** : pour chaque entrée d'entraînement, la sortie désirée est connue (exemple = valeurs d'entrée + sortie désirée).
- **Par renforcement** : une suite d'étapes dont on ne connaît pas la sortie désirée à chaque pas ; on reçoit parfois un retour (récompense ou regret), pas à chaque étape.
- **Non supervisé** : aucune information sur la sortie désirée, seulement les entrées ; on cherche par exemple des groupes (clusters) d'exemples semblables entre eux et dissemblables d'un groupe à l'autre.

Entraînement supervisé (p. 6). Jeu d'exemples $\{(\mathbf{x}_n, d_n)\}$, $n = 1, \dots, N$; par exemple $\mathbf{x}_n = [1,5; 1; \text{NICE}; \dots]$ et $d_n = \text{TRUE}$. Les attributs peuvent être numériques ou symboliques. Un algorithme d'entraînement, *non spécifique au problème*, construit une **hypothèse** $h(\mathbf{x})$: la règle générale tirée des exemples. **Utilisation (p. 7)** : pour une situation nouvelle $\mathbf{x}_{\text{new}} = [2,7; 0; \text{UGLY}; \dots]$, le système propose $h(\mathbf{x}_{\text{new}})$, par exemple NO. Le système *généralise* à des cas jamais vus.

Système entraînable (p. 8)	Apprentissage typique
Réseaux de neurones	les deux (supervisé plus courant)
Réseaux bayésiens, machines à noyau	(non précisé)
Arbres de décision	supervisé
Plus proche voisin	supervisé
K-means	non supervisé
Clustering hiérarchique	non supervisé

Piège

Clustering = non supervisé (pas d'étiquette) ; classification = supervisé.
Un réseau de neurones peut être l'un ou l'autre. Les régressions linéaire et logistique du TP 2 sont supervisées : chaque x a son y .

8. Défis 1 à 5 : entraînement, taille, énergie, vitesse, encombrement

Source : 02_Challenges, p. 9 à 17 du PDF

Les huit défis (p. 9) : temps d'entraînement, taille du modèle, efficacité énergétique, vitesse d'inférence, taille et poids physiques, sûreté, confiance, gouvernance.

Défi 1, temps d'entraînement (p. 10 et 11)

Réseau (classification d'images)	erreur	entraînement
ResNet18	10,76 %	2,5 jours
ResNet50	7,02 %	5 jours
ResNet101	6,21 %	1 semaine
ResNet152	6,16 %	1,5 semaine

Mesuré sur quatre GPU M40. De ResNet18 à ResNet152 : temps multiplié par $10,5/2,5 = 4,2$ pour 4,6 points d'erreur en moins ; de ResNet101 à ResNet152, 3,5 jours de plus pour 0,05 point. **Conclusion : on n'entraîne généralement pas sur l'embarqué.** Parfois on poursuit l'entraînement d'un

modèle déjà entraîné avec de nouvelles données pour l'adapter (ce n'est pas un entraînement depuis zéro).

Défi 2, taille du modèle (p. 12 à 14)

Tendance (p. 12, « montre seulement la tendance ») : en image, AlexNet (2012, 8 couches, 1,4 GFLOP, environ 16 % d'erreur) puis ResNet (2015, 152 couches, 22,6 GFLOP, environ 3,5 %) : taille du modèle $\approx \times 20$. En parole, Deep Speech 1 (2014, 80 GFLOP, 7000 h de données, environ 8 %) puis Deep Speech 2 (2015, 465 GFLOP, 12 000 h, environ 5 %) : opérations d'entraînement $\approx \times 6$ ($465/80 = 5,8$). FLOP : opérations en virgule flottante.

Modèle (p. 13)	prof.	param. (M)	GFLOPS	Mo	top-5 err.
AlexNet	7	58,3	0,725	217	19,2 %
VGG16	16	134,2	15,5	512,2	9,9 %
ResNet18	18	11,7	1,814	23	10,92 %
GoogleNet	22	6	1,57	40	12,9 %
ResNet34	34	21,8	3,664	35	8,58 %
ResNetV1 50	50	25,6	3,858	97,7	7,7 %
ResNetV1 101	101	45	7,57	155	7 %
ResNetV1 152	152	60	11,3	230	6,7 %

Précision top-5 égale 100 moins l'erreur top-5 (la slide donne les deux ; ResNet18 et ResNet34 font 100,02 et 99,98 à cause des arrondis). Plus profond ne veut pas dire plus gros : GoogleNet (22 couches) a 6 M de paramètres, AlexNet (7 couches) en a 58,3 M. Les gros modèles sont durs à distribuer par mise à jour à distance (OTA, p. 14 : un téléphone refuse de télécharger plus de 100 Mo hors Wi-Fi).

Défi 3, efficacité énergétique (p. 15)

AlphaGo : 1920 CPU et 280 GPU, 3000 \$ d'électricité par partie. Sur un appareil embarqué, la consommation vide la batterie ; en centre de données, elle augmente le TCO (*total cost of ownership* : tous les coûts directs et indirects d'achat et d'exploitation sur la durée de vie). Suite au concept 11.

Défi 4, vitesse d'inférence (p. 16)

Besoins : traitement continu (vidéosurveillance), décision rapide (voiture autonome), faible latence (procédés industriels rapides). Problème : les gros modèles sont parfois trop lents et passent mal à l'échelle quand le problème se complique.

Défi 5, taille et poids physiques (p. 17)

Le calcul haute performance vit dans de grandes salles climatisées ; l'embarqué doit être petit et tenir dans des environnements hostiles. AlphaGo (2016) : environ 16 000 kg de matériel, 33 m³ de baies, environ 1 MW (estimations rapportées par la slide). Cerveau humain : 1,3 kg, 1,3 L, 20 W ; corps : 60 à 80 kg, environ 100 W.

Calcul de la slide 17

$1 \text{ MW}/20 \text{ W} = 10^6/20 = 50\,000$ cerveaux humains. Par rapport à la consommation d'un corps entier (100 W) : $10^6/100 = 10\,000$ humains.

9. Défis 6 à 8 : sûreté, confiance, gouvernance

Source : 02_Challenges, p. 18 à 23 du PDF

Défi 6, sûreté (p. 18)

- **Limites matérielles** : risque de panne ou d'erreur dû au calcul limité.
- **Décisions temps réel** : un temps de traitement insuffisant mène à des décisions hâtives ou fausses, surtout en conduite autonome.
- **Interférence et altération** : l'accès physique à l'appareil permet une intervention malveillante.
- **Sécurité « raisonnable »** : on ne pense pas aux menaces futures.

Défi 7, confiance (p. 19 à 22)

Risque de trop ou de trop peu se fier au système, aggravé en embarqué par l'absence d'interface riche : on ne peut pas lui poser de questions comme à un chatbot.

Notion	Définition	Atténuation
Biais	erreur systématique dans la distribution des données ou le fonctionnement du modèle, qui peut léser certains groupes	identifier les groupes concernés, analyser l'impact sur eux
Responsabilité	déterminer qui répond d'un dommage ou d'une décision erronée	cadre légal contre le « trou de responsabilité » : personne ne sait qui répond du dommage
Transparence	pouvoir comprendre et retracer les résultats et la logique du système	outils d'explication, difficiles car ils exigent aussi un effort de l'utilisateur

Autres points (p. 19) : modèles « boîtes noires », vie privée (quelles données sont collectées, stockées, transmises), cohérence du comportement dans tous les scénarios, biais et équité.

Scandale des allocations familiales aux Pays-Bas (p. 21 et 22)

Système algorithmique introduit en 2013 par l'administration fiscale pour détecter les demandes incorrectes et la fraude. La nationalité non néerlandaise était un facteur de risque : **biais**. L'agent chargé de la revue manuelle ne savait pas pourquoi le score était élevé : **transparence**. Les agents ne se sentaient pas responsables de la décision, attribuée au modèle : **responsabilité**, et **dépendance excessive** (over-reliance). Le gouvernement a démissionné le 15 janvier 2021.

Défi 8, gouvernance (p. 23)

Défis : manque de réglementation standardisée pour beaucoup d'applications ; mises à jour et correctifs sans perturbation ; respect de l'éthique, en particulier en surveillance et reconnaissance faciale. Atténuations : cadre légal pour le trou de responsabilité, tests rigoureux en conditions réelles avant déploiement, contrôle humain (difficile en embarqué), dialogue avec utilisateurs, régulateurs et public, cadres éthiques.

Piège

Sûreté (safety) et confiance (trust) se recoupent. Classer selon le mot-clé de la slide : altération physique → sûreté ; boîte noire, biais, vie privée → confiance ; réglementation, mises à jour → gouvernance.

10. Où calculer : cloud ou local

Source : 02_Challenges, p. 24 à 27 du PDF

Mode cloud (p. 24). Des SoC (*System on a Chip*) avec capteurs envoient les données à un centre de calcul qui héberge l'application et l'algorithme d'IA (GPU, TPU). **Mode local (p. 26 et 27).** Dans l'appareil : capteurs, SoC, puce d'accélération IA, algorithme d'IA *allégé*, application.

	Négatif	Positif
Cloud	n'aide pas l'énergie locale limitée ; aggrave le temps réel et le temps de réaction ; bande passante limitée, communication peu fiable	calcul plus puissant ; résout les limites de mémoire
Local	n'aide pas l'énergie locale limitée ; calcul faible ; n'aide pas la mémoire ; compromis entre précision et complexité	meilleur temps de réaction ; la bande passante pose moins de problème

Exemple du mode cloud : Siri (p. 25, d'après Bin Li)

Sur le téléphone : Core Audio fournit le son à un **détecteur de déclenchement vocal** local ; le Voice Trigger Framework fournit l'audio, règle le seuil de détection et envoie les événements de réveil au client Siri. Dans le cloud : le serveur Siri vérifie les premiers mots, peut annuler le déclenchement et journalise. Le serveur peut envoyer une nouvelle configuration du modèle (datapack).

Piège

Aucun des deux modes ne résout la limite d'énergie locale : les deux

listes « négatif » la contiennent. Question classique : « quel mode améliore le temps de réaction ? » (local) ; « lequel aide la mémoire ? » (cloud).

11. Où part l'énergie

Source : 02_Challenges, p. 28 et 33 du PDF

« Modèle plus gros $\Rightarrow$ plus d'accès mémoire $\Rightarrow$ plus d'énergie. »

Opération (Horowitz, 45 nm)	énergie (pJ)	aire (μm^2)
addition 8 bits	0,03	36
addition 16 bits	0,05	67
addition 32 bits	0,1	137
addition flottante 16 bits	0,4	1360
addition flottante 32 bits	0,9	4184
multiplication 8 bits	0,2	282
multiplication 32 bits	3,1	3495
multiplication flottante 16 bits	1,1	1640
multiplication flottante 32 bits	3,7	7700
lecture SRAM 32 bits (8 Ko)	5	
lecture DRAM 32 bits	640	

La slide p. 28 cite aussi le banc de registres 32 bits à 1 pJ et résume : un accès DRAM coûte autant qu'environ 1000 opérations arithmétiques.

Ratios à savoir refaire

DRAM contre addition entière 32 bits : $640/0,1 = 6400$. DRAM contre addition flottante 32 bits : $640/0,9 \approx 711$. DRAM contre SRAM : $640/5 = 128$. Multiplication flottante 32 bits contre multiplication 8 bits : $3,7/0,2 = 18,5$ en énergie et $7700/282 \approx 27$ en surface.

Budget d'une inférence (méthode générale, chiffres de la slide)

Un modèle fait 10^6 multiplications-additions en FP32 et lit ses 10^6 poids

en DRAM. Calcul : $10^6 \times (3,7 + 0,9) = 4,6 \cdot 10^6$ pJ = $4,6 \mu\text{J}$. Mémoire : $10^6 \times 640 = 640 \mu\text{J}$. Total $644,6 \mu\text{J}$, dont 99,3 % pour la DRAM. Si les poids tiennent en SRAM : $4,6 + 5 = 9,6 \mu\text{J}$, environ 67 fois moins. En entiers 8 bits, le calcul tombe à $10^6 \times (0,2 + 0,03) = 0,23 \mu\text{J}$, 20 fois moins qu'en FP32.

Puissance de la DRAM (p. 28) : $P = \frac{1}{2} \sum_i C_i V_i^2 f_i$, la fréquence f_i étant entourée. [rappel général] La puissance est proportionnelle à la fréquence des commutations, donc aux accès : diviser f par 2 divise P par 2 ; diviser V par 2 divise P par 4 ; passer de 1,2 V à 1,0 V multiplie P par $(1,0/1,2)^2 \approx 0,69$.

À retenir

Pour économiser l'énergie en embarqué : garder les poids près du calcul (registres, SRAM), réduire leur nombre (élagage) et leur taille (quantification), préférer l'entier au flottant.

12. Matériel, premières applications, capteurs intelligents

Source : 02_Challenges, p. 29 à 31 du PDF

Taxonomie (p. 29). Matériel généraliste : **CPU** orienté latence (sert un fil d'exécution séquentiel avec une faible latence), **GPU** orienté débit, **TPU**. Matériel spécialisé : **FPGA**, logique programmable ; **ASIC**, logique figée.

[rappel général] Latence : temps pour traiter une donnée ; débit : nombre de données traitées par unité de temps. Un FPGA se reconfigure après fabrication, un ASIC non : il est plus efficace mais son circuit ne change plus.

Premières applications embarquées (p. 30). Détection de mot de réveil (« Ok Google », « Hey Siri ») ; détection d'accident de voiture par fusion de données (micro, capteurs inertiels, GPS) avec alerte des secours ; comptage de pas et détection d'activité (capteurs inertiels et biocapteurs). Ces tâches difficiles ne nous paraissent plus « intelligentes » : ce qui émerveillait hier devient un minimum attendu, comme 5 + 3 pour un enfant de 3 ans puis pour un adulte.

Sensor hubs (p. 31). D'abord des composants séparés : le coprocesseur de mouvement M7 (NXP LPC18A1) à côté de l'Apple A7, aujourd'hui intégré

au SoC des smartphones ; certains restent vendus par les fabricants de capteurs. But commun : des informations utiles pour une très faible consommation. Typiquement un ARM Cortex-M couplé au capteur avec un logiciel préchargé. Va-et-vient dans et hors du SoC : microNPU Ethos U-55 (*Neural Processing Unit*) intégré aux IP ARM.

13. Représentation des nombres et quantification

Source : 02_Challenges, p. 32 et 33 du PDF ; 01_Intro, slide 28

Flottant : $(-1)^S \times (1.M) \times 2^E$ (p. 32).

Format	bits (S, E, M)	total	étendue	précision
FP32	1, 8, 23	32	10^{-38} à 10^{38}	0,000006 %
FP16	1, 5, 10	16	$6 \cdot 10^{-5}$ à $6 \cdot 10^4$	0,05 %
Int32	1, 31	32	0 à $2 \cdot 10^9$	1/2
Int16	1, 15	16	0 à $6 \cdot 10^4$ (slide)	1/2
Int8	1, 7	8	0 à 127	1/2
Virgule fixe	S, I, F			

[rappel général] L'exposant est stocké avec un décalage (127 en FP32, 15 en FP16). Les précisions relatives correspondent à la moitié de l'écart entre deux flottants voisins : $2^{-24} \approx 6 \cdot 10^{-8}$, soit 0,000006 %, en FP32 ; $2^{-11} \approx 4,9 \cdot 10^{-4}$, soit 0,05 %, en FP16. Les valeurs extrêmes calculées avec NumPy : FP16 va de $6,1 \cdot 10^{-5}$ (plus petit normalisé) à 65 504 ; FP32 de $1,18 \cdot 10^{-38}$ à $3,4 \cdot 10^{38}$. Pour les entiers, « 1/2 » s'interprète comme l'erreur d'arrondi maximale d'une demi-unité (interprétation, **À VÉRIFIER**).

Piège

Int8 : 1 bit de signe et 7 bits de magnitude, donc 127 au plus ($2^7 - 1$). Int16 : la slide indique $6 \cdot 10^4$, mais 15 bits de magnitude donnent $2^{15} - 1 = 32\,767$; $6 \cdot 10^4$ correspond à 16 bits sans signe (65 535). Int32 : $2^{31} - 1 \approx 2,1 \cdot 10^9$, cohérent avec la slide. Donner la valeur de la slide si on la demande, en sachant la calculer.

Taille mémoire d'un modèle

Méthode générale : taille = nombre de paramètres $\times$ octets par paramètre (4 en FP32, 2 en FP16, 1 en int8).

VGG16, 134,2 M de paramètres

FP32 : $134,2 \cdot 10^6 \times 4 = 536,8 \cdot 10^6$ octets = 511,9 Mio : c'est la valeur 512,2 de la slide p. 13 (en mébioctets). FP16 : 268,4 Mo. Int8 : 134,2 Mo, quatre fois moins qu'en FP32. C'est l'intérêt de la **quantification** annoncée en 01_Intro, slide 28.

Les autres lignes du tableau ne suivent pas toutes cette règle (ResNet18 : $11,7 \times 4 = 46,8$ Mo en FP32 contre 23 dans la slide ; GoogleNet : 24 Mo contre 40) : format de stockage non précisé, **À VÉRIFIER**.

Énergie par format (p. 33) : à opération égale, le flottant 16 bits coûte moins que le 32 bits (multiplication : 1,1 contre 3,7 pJ, 1640 contre 7700 μm^2), et l'entier 8 bits encore moins (0,2 pJ, 282 μm^2).

14. TP 1 : Python

Source : lab01-exercices.ipynb et lab01-solutions.ipynb, cellules 0 à 71 ; notebook de Mathieu Blondel

Le TP présente les notebooks (Jupyter, Google Colab), puis les bases de Python. Une cellule de code s'exécute avec Ctrl ou Cmd + Entrée ; les variables d'une cellule servent aux suivantes, donc l'ordre d'exécution compte (« Run all » pour tout rejouer dans l'ordre, cellule 10).

À retenir

- Opérateurs : +, *, / (division réelle), ** (puissance), // (division entière).
- Listes : ordonnées, taille variable, types mélangés, indices depuis 0 ; tranche `l[i:j]` de *i* inclus à *j* exclu ; `in`, `len`, `append`.
- Chaînes : indexées et découpées comme des listes ; concaténation par +.
- `if`, `elif`, `else` ; `==` teste l'égalité, `=` affecte ; les blocs sont délimités par l'indentation.

- Boucles while et for; fonctions avec def et return, arguments nommables (multiply(a=3, b=2)).

```
my_list = [1, 2, 3, "apple"]
my_list.append(4)
print(my_list[2], my_list[1:3], my_list[-1], len(my_list))
s = "some text"
print(s[3], s[5:], s + "!")
print(7 / 2, 7 // 2, 2 ** 10, 7 % 3)
print(5 in my_list, "apple" in my_list)
```

Sortie :

```
3 [2, 3] 4 5
e text some text!
3.5 3 1024 1
False True
```

Exercices du TP et solutions

ReLU : $\text{relu}(x) = x$ si $x \geq 0$, 0 sinon. Norme euclidienne par boucle : $\sqrt{\sum x_i^2}$; attendu 5,729746940310715 pour [0,5; -1,2; 3,3; 4,5]. Maximum par boucle. Bonus : tri à bulles, laissé vide dans les deux notebooks ; solution ci-dessous, avec arrêt anticipé qui donne le meilleur cas $O(N)$ du concept 6.

```
import numpy as np

def relu(x):
    if x >= 0:
        return x
    return 0

def euclidean_norm(vector):
    total = 0
    for x in vector:
        total += x * x
    return np.sqrt(total)

def bubble_sort(lst):
    lst = list(lst)
    n = len(lst)
    for i in range(n - 1):
```

```

        swapped = False
        for j in range(n - 1 - i):
            if lst[j] > lst[j + 1]:
                lst[j], lst[j + 1] = lst[j + 1], lst[j]
                swapped = True
        if not swapped:
            break
    return lst

print(relu(-3), relu(2.5))
print(euclidean_norm([0.5, -1.2, 3.3, 4.5]))
print(bubble_sort([1, -3, 3, 2]))

```

Sortie :

```

0 2.5
5.729746940310715
[-3, 1, 2, 3]

```

Piège

Les sorties enregistrées dans `lab01-solutions.ipynb` viennent d'exécutions dans un autre ordre : la cellule 22 affiche `[1, 2, 3, 'apple', 4, 4]` (append lancé deux fois), la cellule 32 affiche une longueur de 4, et la cellule 102 affiche pour `np.linspace(0, 1, 10)` des valeurs de 1 à 10 qui sont celles de `np.logspace(0, 1, 10)`. Ne pas apprendre ces sorties; refaire le calcul.

15. TP 1 : NumPy et Matplotlib

Source : `lab01-solutions.ipynb`, cellules 72 à 158

NumPy stocke des tableaux de nombres d'un seul type (`int32`, `int64`, `float32`, `float64`) et calcule dessus en C : le code est plus court et plus rapide qu'une boucle Python (cellules 73, 89, 112).

À retenir

- Création : `np.array`, `np.zeros((2, 3))`, `np.ones`, `np.arange(3, 5)` (3 et 4), `np.linspace(a, b, n)` (n points de a à b inclus, pas $(b - a)/(n - 1)$), `reshape`.

- `shape` donne les dimensions; `A[i, j]` un élément, `A[i]` une ligne, `A[:, j]` une colonne.
- Opérations élément par élément (`+`, `*`, `np.sin` : fonctions universelles); `np.dot` ou `@` : produit scalaire, ou produit matriciel entre tableaux 2D; `.T` : transposée.
- Masque booléen : `x[x >= 5]`; `axis=0` agrège le long des lignes (résultat par colonne), `axis=1` le long des colonnes (résultat par ligne).

```
import numpy as np
A = np.array([[1, 2, 3], [4, 5, 6]])
print(A.shape, A[1, 2], A[1], A[:, 2])
a, b = np.array([1, 2, 3]), np.array([4, 5, 6])
print(a + b, a * b, np.dot(a, b), a @ b)
x = np.arange(10)
print(x[x >= 5])
print(np.maximum(0, np.array([1, -3, 2.5])))
M = np.array([[0.5, -1.2, 4.5], [-3.2, 1.9, 2.7]])
print(np.sum(M, axis=0), np.sum(M, axis=1))
print(np.sqrt(np.sum(M ** 2, axis=1)))
print(np.linspace(0, 1, 5), np.linspace(0, 10, 51).shape)
```

Sortie :

```
(2, 3) 6 [4 5 6] [3 6]
[5 7 9] [ 4 10 18] 32 32
[5 6 7 8 9]
[1.  0.  2.5]
[-2.7  0.7  7.2] [3.8 1.4]
[4.68401537 4.59782557]
[0.  0.25 0.5  0.75 1.  ] (51,)
```

La ligne `np.maximum` résout l'exercice 2 (ReLU vectorisée); la ligne `np.sqrt(np.sum(M ** 2, axis=1))` résout l'exercice 4 (normes par ligne), laissé vide dans le notebook. Exercice 5 : la moyenne des 4 variables d'iris s'obtient par `X.mean(axis=0)` (non exécuté ici, scikit-learn absent de cette machine).

Matplotlib

`plt.figure()`, `plt.plot(x, y, label=...)`, `plt.xlabel`, `plt.ylabel`, `plt.title`, `plt.legend`, `plt.show()`; `plt.scatter` pour un nuage de

points, coloré par classe (cellules 145 et 147); `plt.imshow` pour une image (un chiffre de digits est un tableau 8×8 , cellule 156). Constat du notebook sur iris : avec les deux premières variables, la classe 0 est séparable linéairement des deux autres (cellule 148).

[rappel général] Softplus : $\ln(1 + e^x)$, version lisse de ReLU, strictement positive et dérivable partout ; $\text{softplus}(0) = \ln 2 \approx 0,693$ alors que $\text{relu}(0) = 0$.

16. TP 2 : régression linéaire, solution analytique

Source : `lab02-solutions.ipynb`, cellules 0 à 4

Données (exercice 1). 51 entrées régulières de 0 à 10 (`np.linspace(0, 10, 51)`, pas 0,2), étiquettes $y = 0,7x + 2$ plus un bruit gaussien (`np.random.randn`). Entrées de test : `np.linspace(0, 10, 101)`.

À retenir

Modèle $y \approx \mathbf{w}^\top \phi(x)$ avec $\phi(x) = [1, x]$. On empile les $\phi(x_i)$ en lignes dans $\mathbf{X}$ (une colonne de 1, puis les x_i). Solution des moindres carrés :

$$\mathbf{w} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}.$$

Prédiction : $\mathbf{y}_{\text{test}} = \mathbf{X}_{\text{test}} \mathbf{w}$.

[rappel général] D'où vient la formule : on minimise $E(\mathbf{w}) = \sum_i (y_i - \mathbf{w}^\top \phi(x_i))^2 = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2$. Le gradient $-2\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\mathbf{w})$ s'annule quand $\mathbf{X}^\top \mathbf{X}\mathbf{w} = \mathbf{X}^\top \mathbf{y}$ (équations normales). La colonne de 1 porte l'ordonnée à l'origine.

À la main : $x = (0, 1, 2, 3)$, $y = (1, 2, 2, 4)$

$$\mathbf{X}^\top \mathbf{X} = \begin{pmatrix} N & \sum x \\ \sum x & \sum x^2 \end{pmatrix} = \begin{pmatrix} 4 & 6 \\ 6 & 14 \end{pmatrix}, \mathbf{X}^\top \mathbf{y} = \begin{pmatrix} \sum y \\ \sum xy \end{pmatrix} = \begin{pmatrix} 9 \\ 18 \end{pmatrix}. \text{ Déterminant}$$

$$4 \times 14 - 6 \times 6 = 20. \text{ Inverse d'une } 2 \times 2 : \frac{1}{20} \begin{pmatrix} 14 & -6 \\ -6 & 4 \end{pmatrix}. \text{ Donc } w_0 = (14 \times 9 - 6 \times 18) / 20 = 18 / 20 = 0,9 \text{ et } w_1 = (-6 \times 9 + 4 \times 18) / 20 = 18 / 20 = 0,9. \\ \text{Droite } y = 0,9 + 0,9x; \text{ prédiction en } x = 5 : 5,4.$$

```

import numpy as np
x = np.array([0.0, 1, 2, 3])
y = np.array([1.0, 2, 2, 4])
X = np.vstack([np.ones(len(x)), x]).T
w = np.linalg.solve(X.T @ X, X.T @ y)
print(X.T @ X)
print(X.T @ y, w)
y_hat = X @ w
rmse = np.sqrt(np.mean((y - y_hat) ** 2))
print(y_hat, round(rmse, 4))
for lam in [0, 1, 10]:
    w = np.linalg.solve(X.T @ X + lam * np.eye(2), X.T @ y)
    print(lam, w.round(4))

```

Sortie :

```

[[ 4.  6.]
 [ 6. 14.]]
[ 9. 18.] [0.9 0.9]
[0.9 1.8 2.7 3.6] 0.4183
0 [0.9 0.9]
1 [0.6923 0.9231]
10 [0.36 0.66]

```

Piège

- `np.vstack([np.ones(N), x]).T` : sans le `.T`, on obtient une matrice $2 \times N$ au lieu de $N \times 2$.
- Le notebook conseille `np.linalg.solve` plutôt que l'inverse explicite : plus stable et moins coûteux. $\mathbf{X}^T \mathbf{X}$ est symétrique définie positive, d'où la piste de la factorisation de Cholesky (cellule 3).
- La solution met un bruit d'écart type 5 (`randn(N)*5`) là où l'énoncé en met 0,8 : la droite estimée s'écarte alors davantage de la vraie ($w_0 = 2, w_1 = 0,7$).

17. TP 2 : régression polynomiale et régularisation

Source : lab02-solutions.ipynb, cellules 5 à 12

Non linéaire (exercices 3 et 4). Données $y = (x - 9)(x - 6)(x - 1)$ plus un bruit d'écart type 5. On garde la même méthode et on change seulement la transformation : $\phi(x) = [1, x, x^2, x^3]$. La matrice Φ a une ligne par exemple et $D + 1 = 4$ colonnes ; `np.vander(x, D+1, increasing=True)` la construit. Le modèle reste *linéaire en $\mathbf{w}$* , c'est pourquoi la même formule marche.

Lignes de Φ et polynôme cible

$x = 0,2$ donne $[1; 0,2; 0,04; 0,008]$ (affichage du notebook), $x = 2$ donne $[1; 2; 4; 8]$. En développant, $(x - 9)(x - 6)(x - 1) = x^3 - 16x^2 + 69x - 54$: les poids idéaux sont $\mathbf{w} = (-54; 69; -16; 1)$, et $y(2) = (-7)(-4)(1) = 28$.

Régularisation (exercice 5). L'estimation au maximum a posteriori (MAP) donne

$$\mathbf{w} = (\Phi^T \Phi + \lambda \mathbf{I})^{-1} \Phi^T \mathbf{y},$$

testée avec $\lambda = 0, 1, 10, 100$ (`np.eye(D+1)` pour $\mathbf{I}$).

[rappel général] Cette formule minimise $\|\mathbf{y} - \Phi \mathbf{w}\|^2 + \lambda \|\mathbf{w}\|^2$ (régression ridge) : le terme en λ pénalise les grands poids. Plus λ est grand, plus les poids rétrécissent vers 0 et plus la courbe s'aplatit : on réduit le surapprentissage, mais un λ trop grand fait sous-apprendre. $\lambda = 0$ redonne les moindres carrés.

Mesuré sur une exécution du code de la solution, graine NumPy fixée

Données du TP (bruit 5), ajustement cubique. Norme de $\mathbf{w}$ et RMSE sur l'entraînement : $\lambda = 0$: 91,8 et 4,98 ; $\lambda = 1$: 64,3 et 7,69 ; $\lambda = 10$: 20,6 et 16,5 ; $\lambda = 100$: 4,1 et 21,4. Les poids fondent, l'erreur d'entraînement monte : ici le modèle cubique est déjà adapté, la régularisation le dégrade. Les valeurs changent avec le tirage.

Ridge à la main, données du concept 16, $\lambda = 1$

$\mathbf{X}^T \mathbf{X} + \mathbf{I} = \begin{pmatrix} 5 & 6 \\ 6 & 15 \end{pmatrix}$, déterminant $75 - 36 = 39$. $w_0 = (15 \times 9 - 6 \times 18) / 39 = 27 / 39 \approx 0,692$; $w_1 = (-6 \times 9 + 5 \times 18) / 39 = 36 / 39 \approx 0,923$. Avec $\lambda = 10$: (0,36; 0,66).

Piège

Avec `np.eye(D+1)`, le notebook régularise aussi le biais w_0 : dans l'exemple, l'ordonnée à l'origine passe de 0,9 à 0,69 alors que la pente bouge à peine. Beaucoup de bibliothèques ne pénalisent pas le biais. La question de la cellule 11 (« comment λ agit-il? ») n'a pas de réponse dans la solution : c'est une question ouverte probable.

18. TP 2 : jeu de données réel et RMSE

Source : `lab02-solutions.ipynb`, cellules 13 à 17

Exercice 6. Jeu diabetes de scikit-learn (`load_diabetes`); découpage aléatoire 80 % entraînement, 20 % test avec `train_test_split(x, y, test_size=0.2, random_state=38)`. [rappel général] Ce jeu compte 442 patients et 10 variables (documentation scikit-learn; le notebook n'affiche pas la forme).

Deux modèles : la formule du concept 16, avec une colonne de 1 ajoutée devant les 10 variables (11 paramètres), et `LinearRegression().fit(x_train, y_train)` puis `.predict(x_test)`. Comparaison par la **RMSE** :

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}.$$

Sorties enregistrées dans le notebook : 56,94023984486523 à la main et 56,940239844865225 avec scikit-learn; la version complétée (`random_state=28`) donne 57,76. Mêmes résultats aux arrondis près : `LinearRegression` résout le même problème des moindres carrés et ajoute elle-même l'ordonnée à l'origine.

RMSE à la main

$y = (3, 5, 2, 7)$, $\hat{y} = (2, 5; 5; 3; 6)$. Erreurs 0,5; 0; -1; 1; carrés 0,25; 0; 1; 1; moyenne (MSE) $2,25/4 = 0,5625$; $\text{RMSE} \sqrt{0,5625} = 0,75$. Sur l'exemple du concept 16 : résidus 0,1; 0,2; -0,7; 0,4, somme des carrés 0,70, MSE 0,175, RMSE 0,418.

Piège

La RMSE s'exprime dans l'unité de y (ici l'indicateur de progression du diabète) ; la MSE dans son carré. On l'évalue sur le jeu de *test*, jamais sur celui qui a servi à calculer $\mathbf{w}$. Le `random_state` fixe le découpage : deux graines donnent deux RMSE différentes (56,94 contre 57,76).

19. TP 2 : régression logistique

Source : `lab02-solutions.ipynb`, cellules 18 à 26

Classification binaire avec un modèle linéaire. Données : deux nuages gaussiens de 100 points, centrés en $(-1, -2)$ (classe 0) et $(0, 0)$ (classe 1) ; test sur une grille 100×100 de $[-5, 5]^2$ pour dessiner la frontière (`np.meshgrid`, `plt.contourf`).

À retenir

- Transformation $\phi(\mathbf{x}) = [1, \mathbf{x}]$; modèle $P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \phi(\mathbf{x}))$ avec $\sigma(a) = 1/(1 + e^{-a})$.
- Descente de gradient : $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} l$, η le pas d'apprentissage, $\mathbf{w}$ initialisé à 0, nombre d'itérations fixé.
- Gradient codé dans la solution : `-Phi.T @ (y*(1-s) - (1-y)*s)` avec `s = sigmoid(Phi @ w)`, qui se simplifie en $\Phi^\top (\sigma(\Phi \mathbf{w}) - \mathbf{y})$.
- Classe prédite : 1 si $\sigma > 0,5$, c'est-à-dire si $\mathbf{w}^\top \phi > 0$: la frontière est la droite $\mathbf{w}^\top \phi(\mathbf{x}) = 0$.

[rappel général] La perte est la log-vraisemblance négative $l(\mathbf{w}) = -\sum_i [y_i \ln \sigma_i + (1 - y_i) \ln(1 - \sigma_i)]$. Avec $\sigma'(a) = \sigma(a)(1 - \sigma(a))$, son gradient vaut $\sum_i (\sigma_i - y_i) \phi_i$. Pas de formule fermée : d'où l'algorithme itératif. En $\mathbf{w} = 0$, tous les σ_i valent 0,5 et $l = N \ln 2$.

a	-2	-1	0	1	2	3	4,6
$\sigma(a)$	0,1192	0,2689	0,5	0,7311	0,8808	0,9526	0,9900

Un pas de descente à la main

Points (0, 0), (1, 0) de classe 0 et (0, 2), (2, 2) de classe 1 ; Φ a pour lignes [1, 0, 0], [1, 1, 0], [1, 0, 2], [1, 2, 2]. En $\mathbf{w} = 0$: $\sigma = 0,5$ partout, donc $\sigma - y = (0,5; 0,5; -0,5; -0,5)$. Gradient $\Phi^T(\sigma - y)$: biais $0,5 + 0,5 - 0,5 - 0,5 = 0$; $x_1 : 0,5 \times 1 - 0,5 \times 2 = -0,5$; $x_2 : -0,5 \times 2 - 0,5 \times 2 = -2$. Avec $\eta = 0,1$: $\mathbf{w} = (0; 0,05; 0,2)$. La perte passe de $4 \ln 2 \approx 2,773$ à $2,399$.

```
import numpy as np

def sigmoid(a):
    u = np.exp(-np.abs(a))
    return np.where(a > 0, 1 / (1 + u), u / (1 + u))

Phi = np.array([[1.0, 0, 0], [1, 1, 0], [1, 0, 2], [1, 2, 2]])
y = np.array([0.0, 0, 1, 1])
w = np.zeros(3)
eta = 0.1
for it in range(3):
    grad = Phi.T @ (sigmoid(Phi @ w) - y)
    w = w - eta * grad
    print(it, grad.round(4), w.round(4))
pred = np.where(sigmoid(Phi @ w) > 0.5, 1, 0)
print(pred, np.sum(pred != y))
```

Sortie :

```
0 [ 0. -0.5 -2. ] [0.  0.05 0.2 ]
1 [ 0.2336 -0.2426 -1.5577] [-0.0234  0.0743  0.3558]
2 [ 0.3702 -0.0917 -1.2733] [-0.0604  0.0834  0.4831]
[0 1 1 1] 1
```

Après trois pas, le point (1, 0) est encore mal classé : il faut plus d'itérations.

Piège

- **Sigmoïde stable** (cellule 22) : e^{-a} déborde pour a très négatif (e^{800} vaut inf en float64, avec un avertissement). La version du TP ne calcule que $u = e^{-|a|} \leq 1$ et renvoie $1/(1+u)$ si $a > 0$, $u/(1+u)$ sinon.
- **Comptage d'erreurs** : la solution écrit `num_errs = np.sum(pred-y)`. Les erreurs +1 (faux positifs) et -1 (faux négatifs) se compensent : pour `pred = (1, 0, 1, 0)` et `y = (0, 1, 1, 0)`, on obtient 0 alors qu'il y a

2 erreurs. Écrire `np.sum(pred != y)`. Sur une exécution de la solution (graine fixée), on a mesuré 28 erreurs sur 200 (86 % de bonnes réponses) alors que `np.sum(pred=y)` vaut -20 .

- Le gradient est une *somme* sur les 200 points, pas une moyenne : le pas $\eta = 0,1$ agit donc 200 fois plus fort qu’avec une moyenne.

Exercice facultatif (cellule 26) : dériver le gradient de la perte régularisée et entraîner un modèle logistique non linéaire régularisé pour provoquer sous- et surapprentissage. [rappel général] Avec la pénalité $\frac{\lambda}{2} \|\mathbf{w}\|^2$, le gradient devient $\Phi^T(\sigma - \mathbf{y}) + \lambda \mathbf{w}$.

20. Ce qui reste au programme

Source : 01_Intro, slide 4; fiche CLAUDE.md, « Programme »

Le programme annonce ces thèmes, sans support disponible au 2026-09-28. Ils ne sont pas développés ici ; à compléter dès que les slides arrivent sur Teams.

1. Matériel cible pour l’IA embarquée (commencé en 02_Challenges, p. 29 à 33).
2. Classification et **métriques d’évaluation**.
3. Clustering et réduction de dimension.
4. Réseaux de neurones ; bruit et apprentissage ; surapprentissage, early stopping, backtracking ; découpage entraînement, validation, test.
5. Réseaux convolutifs ; système de reconnaissance de formes embarqué.
6. Réseaux récurrents et prédiction.
7. Auto-encodeurs.
8. Apprentissage profond, réseaux génératifs, transformers.
9. Adaptation des réseaux de neurones à l’embarqué (quantification et élagage, annoncés en 01_Intro, slide 28).

Limite du midterm dans cette liste : **À VÉRIFIER**. Sujets des années précédentes : promis par l’enseignant le 2026-09-24, pas encore reçus.

21. Méthode d'examen

Source : format : fiche CLAUDE.md ; types de questions : déduits des supports

Format annoncé : QCM et questions ouvertes, 50 points, 2 heures. Barème détaillé et documents autorisés : **À VÉRIFIER**. Tant que les sujets des années précédentes manquent, les types ci-dessous sont une hypothèse de travail tirée des slides et des TP

Question probable	Où réviser
contraintes critiques d'une application	concept 1 (trois exemples)
définitions IA, ML, DL, génératif ; rationalité	concepts 2 et 3
TinyML, EdgeAI, quantification, élagage	concept 5
définition de O, preuve avec c et N_0	concept 6
pièces, tris, comparaison de croissances	concept 6
supervisé, non supervisé, renforcement	concept 7
les huit défis, cloud ou local	concepts 8 à 10
ratios d'énergie, budget d'une inférence	concept 11
bits d'un format, taille d'un modèle	concept 13
lire du code Python ou NumPy, prédire la sortie	concepts 14 et 15
équations normales, ridge, RMSE à la main	concepts 16 à 18
sigmoïde, frontière, un pas de gradient	concept 19

À retenir

1. QCM : relire chaque option contre la slide ; attention aux options ab-

- solues (toujours, jamais) et aux inversions (CPU latence, GPU débit).
2. Question ouverte : définir, citer la slide (exemple chiffré), conclure pour l'embarqué.
 3. Calcul : écrire la formule, poser les unités (pJ, Wh, octets), vérifier l'ordre de grandeur.
 4. Régression : $\mathbf{X}^\top \mathbf{X}$ et $\mathbf{X}^\top \mathbf{y}$ d'abord, puis l'inverse 2×2 par la formule du déterminant.

Piège

- Oublier la colonne de 1 dans $\mathbf{X}$ (pas d'ordonnée à l'origine).
- Confondre MSE et RMSE, ou calculer la RMSE sur les données d'entraînement.
- Confondre la classe O et le nombre réel d'opérations (pièces, tri).
- Compter les erreurs par `np.sum(pred - y)`.

22. Formulaire

À retenir

Note finale	examen +0,2× midterm + quiz ; réussite à 24
Signature	midterm $\geq 20/50$
$T(N) = O(F(N))$	$\exists c, N_0 > 0, T(N) \leq cF(N)$ pour $N > N_0$
Pièces	force brute $N - 1$; tiers $\lceil \log_3 N \rceil$
Tris	bulles $O(N^2)$, mémoire $O(1)$; merge $O(N \log N)$, mémoire $O(N)$
Énergie (pJ)	add. 32b 0,1 ; add. FP32 0,9 ; mult. FP32 3,7 ; mult. 8b 0,2 ; SRAM 5 ; DRAM 640
DRAM	$P = \frac{1}{2} \sum C_i V_i^2 f_i$
Formats	FP32 (1, 8, 23) ; FP16 (1, 5, 10) ; int8 de 0 à 127
Taille modèle	paramètres × octets (4, 2, 1)
Moindres carrés	$\mathbf{w} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$
Ridge	$\mathbf{w} = (\mathbf{\Phi}^\top \mathbf{\Phi} + \lambda \mathbf{I})^{-1} \mathbf{\Phi}^\top \mathbf{y}$
Inverse 2×2	$\begin{pmatrix} a & b \\ c & d \end{pmatrix}^{-1} = \frac{1}{ad-bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$
RMSE	$\sqrt{\frac{1}{N} \sum (y_i - \hat{y}_i)^2}$
Sigmoïde	$\sigma(a) = 1/(1 + e^{-a})$; $\sigma(0) = 0,5$; $\sigma' = \sigma(1 - \sigma)$
Logistique	$\nabla l = \mathbf{\Phi}^\top (\sigma(\mathbf{\Phi} \mathbf{w}) - \mathbf{y})$; $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla l$
Perte en $\mathbf{w} = 0$	$N \ln 2$ ($\ln 2 \approx 0,693$)

Ordres de grandeur

ChatGPT	$2 \cdot 10^9$ requêtes $\times$ 0,34 Wh = $0,68 \cdot 10^9$ Wh par jour
GPT-4 (entraînement)	62 318 800 kWh, 100 jours, environ 48 fois GPT-3
AlphaGo	1920 CPU, 280 GPU, environ 1 MW, 3000 \$ par partie
Cerveau	1,3 kg, 20 W : 1 MW = 50 000 cerveaux
ResNet18 à 152	erreur 10,76 à 6,16 % ; 2,5 jours à 1,5 semaine
VGG16	134,2 M paramètres, 512 Mo

NumPy	
<code>linspace(a,b,n)</code>	n points, pas $(b - a)/(n - 1)$
<code>axis=0</code> / <code>axis=1</code>	résultat par colonne / par ligne
<code>A[:, j]</code> , <code>A.T</code> , <code>@</code>	colonne, transposée, produit matriciel
